

COL1000: Introduction to Programming

Lecture 03-07

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

Lecture 03

Review of last lecture

1. Printing outputs:

`print( arg1, arg2, ... ) :`

prints `arg1 arg2 ...` (with a space between `arg1` and `arg2`)

`arg1`, `arg2` etc can either be integers, floats, booleans, strings, or predefined variables.

Program

```
print("Hello")  
  
x = "COL"  
  
print("Hello", x, 1000)
```

Output

```
Hello  
Hello COL 1000
```

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

Program

```
print (type (100) )  
print (type (100.0) )  
print (type ("100") )  
print (type (100>90) )
```

Output

```
<class 'int'>  
<class 'float'>  
<class 'string'>  
<class 'bool'>
```

Note how Python executes each line. It first executes the innermost function (for example, `type("100")`). The result of this function (in this case, the result is `<class 'string'>`) is passed to `print`, which then displays it.

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

variables to store data: `var_name = value`

Program

```
age = 22
age = 50.2
print(age)
age = True
print(age)
```

The value for `var_name` can be reassigned. Python takes the latest value at that point.

Output

```
50.2
True
```

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

variables to store data: `var_name = value`

Different operations can be performed on data

Arithmetic operations on integers and floats:

- addition : `+` (output is integer if all operands are integer, else float)
- subtraction: `-` (output is integer if all operands are integer, else float)
- multiplication: `*` (output is integer if all operands are integer, else float)
- division: `/` (output is always a float)

- integer division: `//`
(outputs floor of division, is integer if both operands are integer, else float)

- remainder: `%`
(outputs remainder, is integer if both operands are integer, else float)

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

variables to store data: `var_name = value`

Different operations can be performed on data

Boolean operations on integers and floats (output is always boolean):

- equality check: `==`

True if the two inputs are equal

(be careful when using equality check with floats, and remember to use `==`, not `=`)

- not equal check: `!=`

True if the two inputs are not equal

- Comparison: `>`, `<`, `>=`, `<=`

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

variables to store data: `var_name = value`

Different operations can be performed on data

Logical operations on booleans:

- not: `not`

True is mapped to False, and False is mapped to True

- and: `and`

True only if all the operands are True

- or: `or`

False only if all the operands are False

`and`, `or` can also be applied to strings, but the behaviour is not what you expect it to be.

Since our focus is not to learn all details of Python, we will not expect you to know such details.

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

variables to store data: `var_name = value`

Different operations can be performed on data

Logical operations on booleans:

Exercise:

`x` and `y` are two booleans. What does the following expression compute:

```
(x and (not y)) or ((not x) and y)
```

True if `x` and `y` are different, False if `x` and `y` are same.

Review of last lecture

2. Managing data:

Primitive data types: int, float, string, bool

variables to store data: `var_name = value`

Different operations can be performed on data

Operations on strings:

- concatenation: `+`

all operands must be strings, they are concatenated (without any space)

- repeat a string multiple times: `*`

one operand is a string `s`, another is an integer `k`, and output is `s` repeated `k` times

Review of last lecture

3. Reading inputs from user:

`input("your prompt for the user :")`

Input is always a string, you will need to convert it to integer/float/bool if needed.

```
name = input("Enter your name: ")
print("Hello, " + name + "!!")

age = int(input("Enter your age: "))
```

Python executes the innermost function.

(for example, `input("Enter your age: ")`).

The result of this function (in this case, the result is whatever the user enters here) is passed to `int`, which then converts it to an integer and assigns it to `age`.

Review of last lecture

Branching based on conditions:

What if we need to make some decisions, and execute certain parts based on our decisions?

```
if <condition>:  
    <expression>  
    <expression>  
    ...
```

```
if <condition>:  
    <expression>  
    <expression>  
    ...  
else:  
    <expression>  
    <expression>  
    ...
```

Note the indentation,
this is necessary

```
if <condition>:  
    <expression>  
    <expression>  
    ...  
elif <condition>:  
    <expression>  
    <expression>  
    ...  
else:  
    <expression>  
    <expression>  
    ...
```

- <condition> has a value True or False
- evaluate expressions in that block if <condition> is True

Exercises

1. Write a program that takes as input three integers from the user, and prints them all in one line, with a space in between. For example, if the inputs are 23, 45 and 111, the output should be 23 45 111
2. Write a program that takes as input three integers from the user, and prints them all in one line, **without** a space in between. For example, if the inputs are 23, 45 and 111, the output should be 2345111
3. Write a program where user enters five digits in the range 1-9, and the program outputs twice the number with these five digits. For example, if the user entered 1, 4, 2, 6, 3, then the output should be 28526.

Finding the largest of four distinct numbers

Program

```
a = 6.6
b = 3.1
c = 2.4
d = 2

print("the numbers are:" a, b, c, d)

if ( (a > b) and (a > c) and (a > d) ):
    print("the largest number is", a)
elif ( (b > a) and (b > c) and (b > d) ):
    print("the largest number is", b)
elif ( (c > a) and (c > b) and (c > d) ):
    print("the largest number is", c)
else:
    print("the largest number is", d)
```

How many
comparisons if d
is the largest?

9

Can we reduce number
of comparisons?

Finding the largest of four distinct numbers

Program

```
a = 6.6
b = 3.1
c = 2.4
d = 2

        (a > b) and (a > c ) and (a > d)
if ( _____ ) :
    print("the largest number is", a)
        (b > c ) and (b > d)
elif ( _____ ) :
    print("the largest number is", b)
        (c > d)
elif ( _____ ) :
    print("the largest number is", c)

else:
    print("the largest number is", d)
```

How many
comparisons if d
is the largest?

6

Can we reduce number
of comparisons?

Finding the largest of four distinct numbers: Approach 2

Program

```
a = 6.6
b = 3.1
c = 2.4
d = 2

max = a
if ( b > max ) :
    max = b
if ( c > max ) :
    max = c
if ( d > max ) :
    max = d

print("the largest number is", max)
```

How many
comparisons?

Exercise

Write a program to compute the **second largest out of **three distinct** numbers.**

```
quiz1 = 5.6  
quiz2 = 5.0  
quiz3 = 5.2  
.  
.  
.
```

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
c = int(input("Enter third number: "))  
  
if (((a<b) and (a>c)) or ((a>b) and (a<c))):  
    print(a,"is the second largest number")  
  
elif (((b>a) and (b<c)) or ((b<a) and (b>c))):  
    print(b,"is the second largest number")  
  
else:  
    print(c,"is the second largest number")
```

Exercise: Guess the output of the program

```
x = 15  
  
if (x > 10) :  
  
if (x < 20) :  
    if (x % 2 == 0):  
        print("Output1")  
    else:  
        print("Output2")  
else:  
    if (x % 2 == 0):  
        print("Output3")  
    else:  
        print("Output4")  
else:  
    print("Output5")
```

Error

Code block inside the first
'if' is empty



Exercise: Are these two programs equivalent ?

```
a = input("Enter number: ")
```

```
b = int(a)
```

```
if ( (b % 3) == 0):
```

```
    print("remainder is 0")
```

```
elif ( (b % 3) == 1):
```

```
    print("remainder is 1")
```

```
else:
```

```
    print("remainder is 2")
```

```
a = input("Enter number: ")
```

```
b = int(a)
```

```
if ( (b % 3) == 0):
```

```
    print("remainder is 0")
```

```
if ( (b % 3) == 1):
```

```
    print("remainder is 1")
```

```
if ( (b % 3) == 2):
```

```
    print("remainder is 2")
```

Yes !

Exercise: Are these two programs equivalent ?

```
a = input("Enter number: ")
```

```
b = int(a)
```

```
if ( (b % 3) == 0):
```

```
    print("remainder is 0")
```

```
if ( (b % 3) == 1):
```

```
    print("remainder is 1")
```

```
else:
```

```
    print("remainder is 2")
```

```
a = input("Enter number: ")
```

```
b = int(a)
```

```
if ( (b % 3) == 0):
```

```
    print("remainder is 0")
```

```
if ( (b % 3) == 1):
```

```
    print("remainder is 1")
```

```
if ( (b % 3) == 2):
```

```
    print("remainder is 2")
```

No !

Exercises

Write a program that takes four integers as input, and prints the largest odd number out of the four. If all numbers are even, then it prints the smallest of the four numbers.

How to decompose the problem into simpler sub-problems?

Sub-problem 1: identify if all numbers are even

Sub-problem 2: find smallest of the four numbers

Sub-problem 3: find largest of the **odd** numbers

Exercises

Sub-problem 1: identify if all numbers are even

```
a = input("Enter first number")
b = input("Enter second number")
c = input("Enter third number")
d = input("Enter fourth number")
```

```
a_int = int(a)
b_int = int(b)
c_int = int(c)
d_int = int(d)
```

```
all_even = True
```

```
if((a_int%2) == 1):
    all_even = False
```

```
if((b_int%2) == 1):
    all_even = False
```

```
if((c_int%2) == 1):
    all_even = False
```

```
if((d_int%2) == 1):
    all_even = False
```

```
all_even = ((a_int%2 == 0) and (b_int%2 == 0) and (c_int%2 == 0) and (d_int%2 == 0))
```

Exercises

Sub-problem 2: find smallest of the four numbers

```
if(all_even):
```

```
    min = a_int
```

```
    if (b_int < min):
```

```
        min = b_int
```

```
    if(c_int < min):
```

```
        min = c_int
```

```
    if (d_int < min):
```

```
        min = d_int
```

```
    print("all numbers are even, smallest one is", min)
```

```
else:
```


Exercises

Sub-problem 3: find largest of the odd numbers

```
if(all_even):
```

```
.....
```

```
else:
```

```
    largest_odd = None
```

```
    if ((a_int % 2) == 1):
```

```
        largest_odd = a_int
```

```
    if ((b_int % 2) == 1):
```

```
        if ((largest_odd is None) or (b_int > largest_odd)) :  
            largest_odd = b_int
```

```
    if ((c_int % 2) == 1):
```

```
        if ((largest_odd is None) or (c_int > largest_odd)):  
            largest_odd = c_int
```

```
    if ((d_int % 2) == 1):
```

```
        if ((largest_odd is None) or (d_int > largest_odd)):  
            largest_odd = d_int
```


Lecture 04

Iteration: While loops

Recap : What is the output of this program?

```
a = 12
b = 15
c = 10

if (a > b) :
    if ((c%4) == 0):
        print("output 1")
else:
    print("output 2")
```

Recap : What is the output of this program?

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
c = int(input("Enter third number: "))
```

```
d = 0
```

```
if (a <= b):  
    if (a <= c):  
        d = a  
    else:  
        d = c
```

```
else:  
    if (b <= c):  
        d = b  
    else:  
        d = c
```

Value of d here?

```
if (a % 2 != 0):  
    d = a
```

Value of d here?

```
if ((b % 2 != 0) and (b > d)):  
    d = b
```

Value of d here?

```
if ((c % 2 != 0) and (c > d)):  
    d = c
```

Value of d here?

```
print(d)
```

Iteration: the repetition of a process

Python allows repetition of code segments
using **while** / **for** loops

Syntax

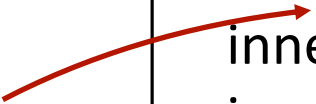
```
while <condition> :  
    inner code block  
    inner code block  
    inner code block
```

```
outer code block 1
```

boolean



Indentation



Inner code block is executed as long as condition is True

While loop : syntax

Outer code block 1

boolean

while <condition> :

Inner code block

Outer code block 2

Inner code block is executed as long as condition is True

Indentation

Example: iteration using while

```
i = 1
```

```
while (i < 5):
```

```
    print(i)
```

```
    i = i+1
```

Output

```
1
```

```
2
```

```
3
```

```
4
```

Indentation

What is the output of the following code

```
i = 1  
  
while (i < 5):  
    print('*' * i)  
    i = i+1
```

Indentation

Output

```
*  
**  
***  
****
```


Exercise

Write a program that takes as input an integer, and outputs the following pattern

Example output for n = 5

```
*
***
*****
*****
*****
```

```
n = int(input("enter number: "))
```

```
i = 1
```

```
while (i <= n):
```

```
    s = " " * (n-i)
```

```
    t = "*" * (2*i - 1)
```

```
    u = s + t
```

```
    print(u)
```

```
    i = i + 1
```


What does this program compute?

```
n = int(input("enter positive integer: "))  
  
i = 0  
  
answer = 0  
  
while (i < n):  
    answer = answer + n  
    i = i + 1  
  
print(answer)
```

If input is positive integer n , it computes square of the number (that is, $n*n$).

What happens if the input is a negative number?

Exercise

Write a program that takes as input an integer, and outputs the following pattern

Example output for $n = 5$

```
*****  
*   *  
*   *  
*   *  
*   *  
*****
```

What does this program compute?

```
n = input("Enter number : ")
```

```
i = 0
```

```
while (i < n):
```

```
    print("*"*n)
```

```
    i = i + 1
```

Output

```
TypeError: '<' not supported between instances of 'int' and 'str'
```

What does this program compute?

```
n = int(input("Enter number : "))
```

```
i = 0
```

```
while (i < n):
```

```
    print("*"*n)
```

```
    i = i + 1
```

Output

```
Enter number: 5
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

What does this program compute?

```
n = int(input("Enter first number : "))
m = int(input("Enter second number : "))

i = 0

while (i < n):
    j = 0

    while(j < m):
        print(i, j)
        j = j+1

    i = i+1
```

Output

```
Enter first number: 2
Enter second number: 3
0 0
0 1
0 2
1 0
1 1
1 2
```

Check out <https://pythontutor.com/visualize.html#mode=edit>

What does this program compute?

```
n = int(input("Enter first number : "))
m = int(input("Enter second number : "))

i = 0
j = 0

while (i < n):

    while(j < m):
        print(i, j)
        j = j+1

    i = i+1
```

Output

```
Enter first number: 2
Enter second number: 3
0 0
0 1
0 2
1 0
1 1
1 2
```

Check out <https://pythontutor.com/visualize.html#mode=edit>

Exercise: Multiplication table

Write a program that takes as input an integer n , and prints the multiplication table of n (from $n*1$ to $n*n$).

```
n = int( input("enter number: "))
```

```
i = 1
```

```
while (i <= n):
```

```
    x = n*i
```

```
    print(n,"*",i,"=", x)
```

```
    i = i+1
```

Output

```
enter number: 15
```

```
15 * 1 = 15
```

```
15 * 2 = 30
```

```
15 * 3 = 45
```

```
15 * 4 = 60
```

```
15 * 5 = 75
```

```
15 * 6 = 90
```

```
15 * 7 = 105
```

```
15 * 8 = 120
```

```
15 * 9 = 135
```

```
15 * 10 = 150
```

```
15 * 11 = 165
```

```
15 * 12 = 180
```

```
15 * 13 = 195
```

```
15 * 14 = 210
```

```
15 * 15 = 225
```


Exercise: Square root

Write a program that takes as input a positive integer n , and prints the largest y such that $y*y \leq n$

```
n = int(input("Enter a positive integer: "))  
  
y = 0  
while ((y + 1) * (y + 1) <= n):  
    y = y + 1  
  
print("The largest y such that y*y <= n is:", y)
```

Output

```
enter number: 15  
3
```

Output

```
enter number: 32  
5
```

Output

```
enter number: 49  
7
```


Exercise: Greatest common divisor

Write a program that takes as input two positive integers a,b, and prints the largest g such that g divides both a and b.

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))

g = 1
gcd = 1

while ((g <= a) and (gcd <= b)):
    # the gcd has to be a number between 1 and min(a,b)
    # we will search this space sequentially,
    # starting with 1

    if ((a%g == 0) and (b%g == 0)):
        # if g divides both
        gcd = g

    g = g+1

print("GCD is",gcd)
```

Output

```
enter number: 15
enter number: 33
GCD is 3
```

Output

```
enter number: 49
enter number: 21
GCD is 7
```

Exercise: Greatest common divisor

Write a program that takes as input two positive integers a,b, and prints the largest g such that g divides both a and b.

```
a = int(input("first number : "))
b = int(input("second number : "))
```

```
g = 1
gcd = 1
```

```
while ((g <= a) and (gcd <= b)):
```

```
    if ((a%g == 0) and (b%g == 0)):
        gcd = g
```

```
    g = g+1
```

```
print("GCD is",gcd)
```

```
a = int(input("first number : "))
b = int(input("second number : "))
```

```
g = 1
gcd = 1
```

```
while ((g <= a) and (gcd <= b)):
```

```
    if ((a%g == 0) and (b%g == 0)):
        gcd = g
```

```
    g = g+1
```

```
    print("GCD is",gcd)
```

```
a = int(input("first number : "))
b = int(input("second number : "))
```

```
g = 1
gcd = 1
```

```
while ((g <= a) and (gcd <= b)):
```

```
    if ((a%g == 0) and (b%g == 0)):
        gcd = g
        print("GCD is",gcd)
```

```
    g = g+1
```

What are the outputs of the above three programs?

Lecture 05

While loops continued ...

Greatest common divisor (GCD)

Write a program that takes as input two positive integers a,b, and prints the largest g such that g divides both a and b.

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))

g = 1
gcd = 1

while ((g <= a) and (g <= b)):
    # the gcd has to be a number between 1 and min(a,b)
    # we will search this space sequentially,
    # starting with 1

    if ((a%g == 0) and (b%g == 0)):
        # if g divides both
        gcd = g

    g = g+1

print("GCD is",gcd)
```

Output

```
enter number: 15
enter number: 33
GCD is 3
```

Output

```
enter number: 49
enter number: 21
GCD is 7
```

Try this program with large (nine digit) numbers

Greatest common divisor (GCD): Testing

Write a program that takes as input three positive integers a,b,g, and outputs “Yes” if g is the GCD of a and b.

```
a = int(input("Enter a : "))
b = int(input("Enter b : "))
g = int(input("Enter g : "))

i = g+1

check = True

# check g divides both a and b

if(a%g != 0):
    check = False

if(b%g != 0):
    check = False
```

```
# check g is the largest number
# that divides both a and b

while((i <= a) and (i <= b)):
    if((a%i == 0) and (b%i == 0)):
        check = False

    i = i+1

if(check):
    print("Yes")
else:
    print("No")
```

Do we need to continue this loop when check = False?

Exiting early out of a loop

Exit loop if some condition holds
using **break** statement

```
while <condition1> :
```

```
    Code block 1
```

```
    if <condition2> :
```

```
        code block 2
```

```
        break
```

```
        code block 3
```

```
    Code block 4
```

If condition2 holds, then code block 2 is executed, but not code block 3.

Output of this program

```
n = 5
i = 1

while (i<=n):
    print("Output 1")

    if(i*i > n):
        print("Output 2")
        break
        print("Output 3")

    print("Output 4")
    i = i+1

print("Output 5")
```

How many lines of output does this program produce?

What is the output of this program?

Greatest common divisor (GCD): Testing

Write a program that takes as input three positive integers a,b,g, and outputs “Yes” if g is the GCD of a and b.

```
a = int(input("Enter a : "))
b = int(input("Enter b : "))
g = int(input("Enter g : "))

i = g+1

check = True

# check g divides both a and b

if(a%g != 0):
    check = False

if(b%g != 0):
    check = False
```

```
# check g is the largest number
# that divides both a and b

while((i <= a) and (i <= b)):

    if((a%i == 0) and (b%i == 0)):

        check = False
        break

    i = i+1

if(check):
    print("Yes")
else:
    print("No")
```


Exercise: Primality testing

A positive number n is prime if the only numbers that divide it are 1 and n . Write a program that takes as input n , and outputs “Yes” if n is prime.

```
n = int(input("enter number: "))  
  
i = 2  
  
prime = True  
  
while(i < n):  
    if (n%i == 0):  
        prime=False  
        break  
  
    i=i+1  
  
if(prime):  
    print(n,"is prime")  
else:  
    print(n,"is not prime")
```

Output

```
enter number: 15  
No
```

Output

```
enter number: 31  
Yes
```

Can we make this more efficient?

Lecture 06

For loops

Summary of Lectures 02 - 05

Python executes statements in order of control flow

Statements encountered in the course so far :

print statement

```
print(arg1, arg2, ... )
```

string/float/integer/boolean value/variable

Complex expression involving
string/float/integer/boolean variables/values

Summary of Lectures 02 - 05

Python executes statements in order of control flow

Statements encountered in the course so far :

Variable assignments

var_name =

string/float/integer/boolean value

Complex expression involving
string/float/integer/boolean variables/values

input("enter input : "),
str(...), int(...), float(...), bool(...)

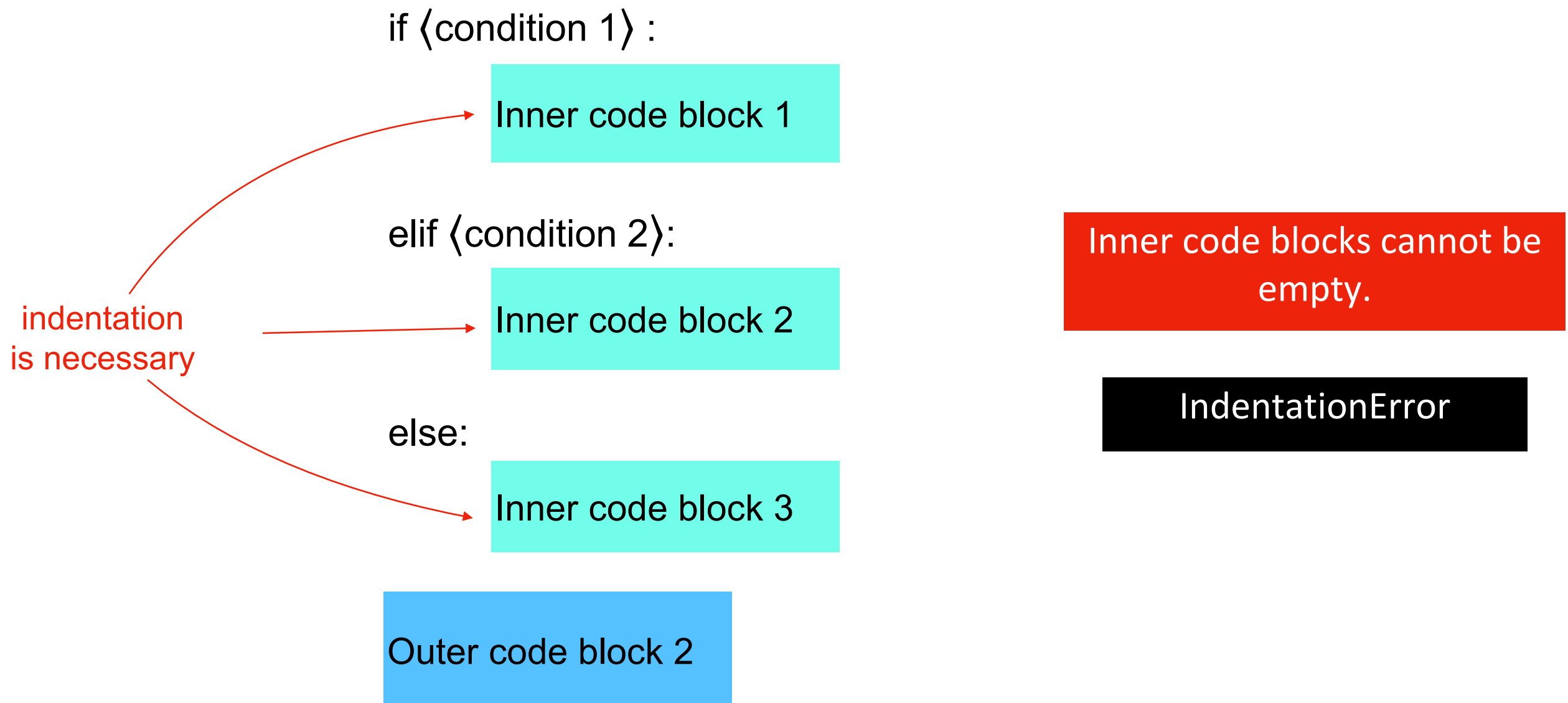
Python evaluates the right-hand side, then sets the name on the left to that result
(replacing any previous value).

Summary of Lectures 02 - 05

Python executes statements in order of control flow

Statements encountered in the course so far :

Conditional branching - if ... elif ... else

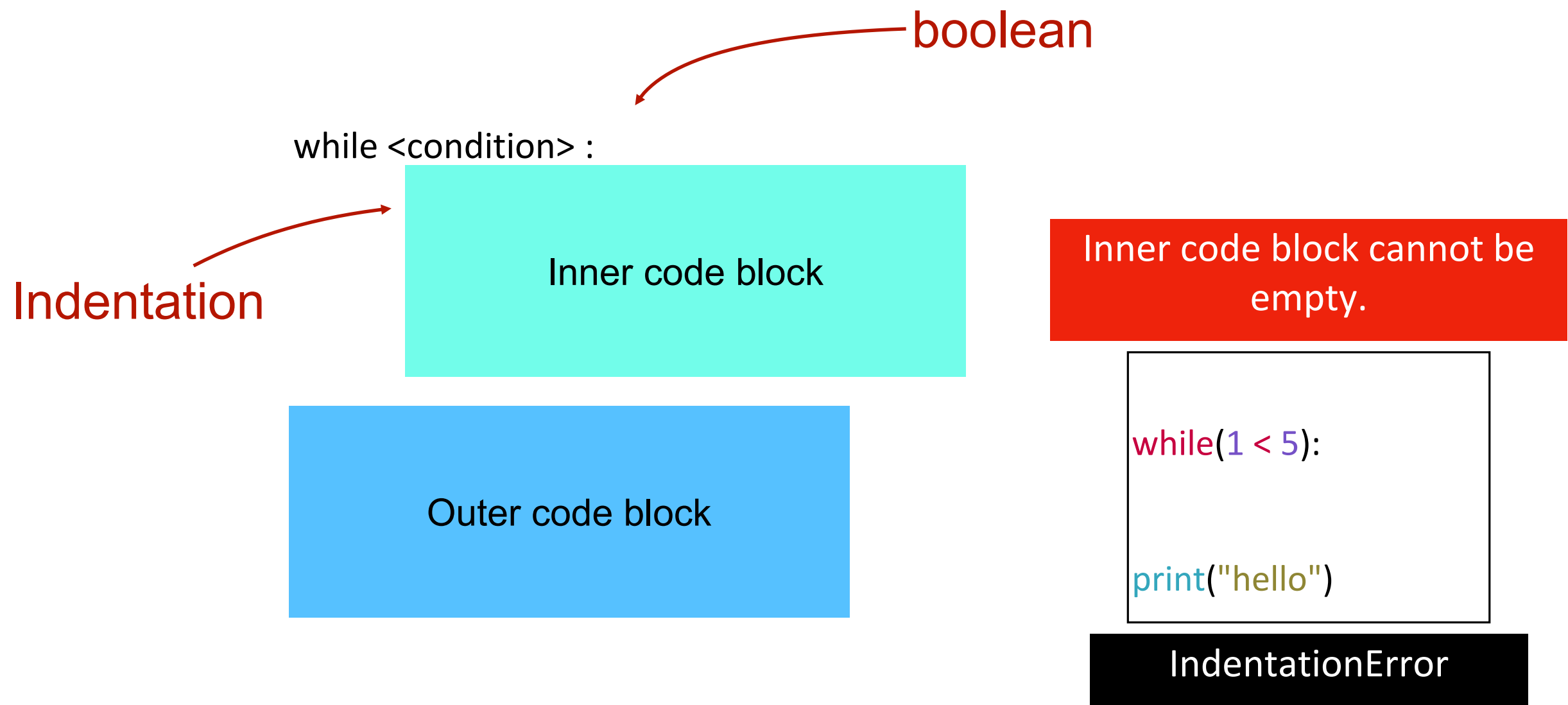


Summary of Lectures 02 - 05

Python executes statements in order of control flow

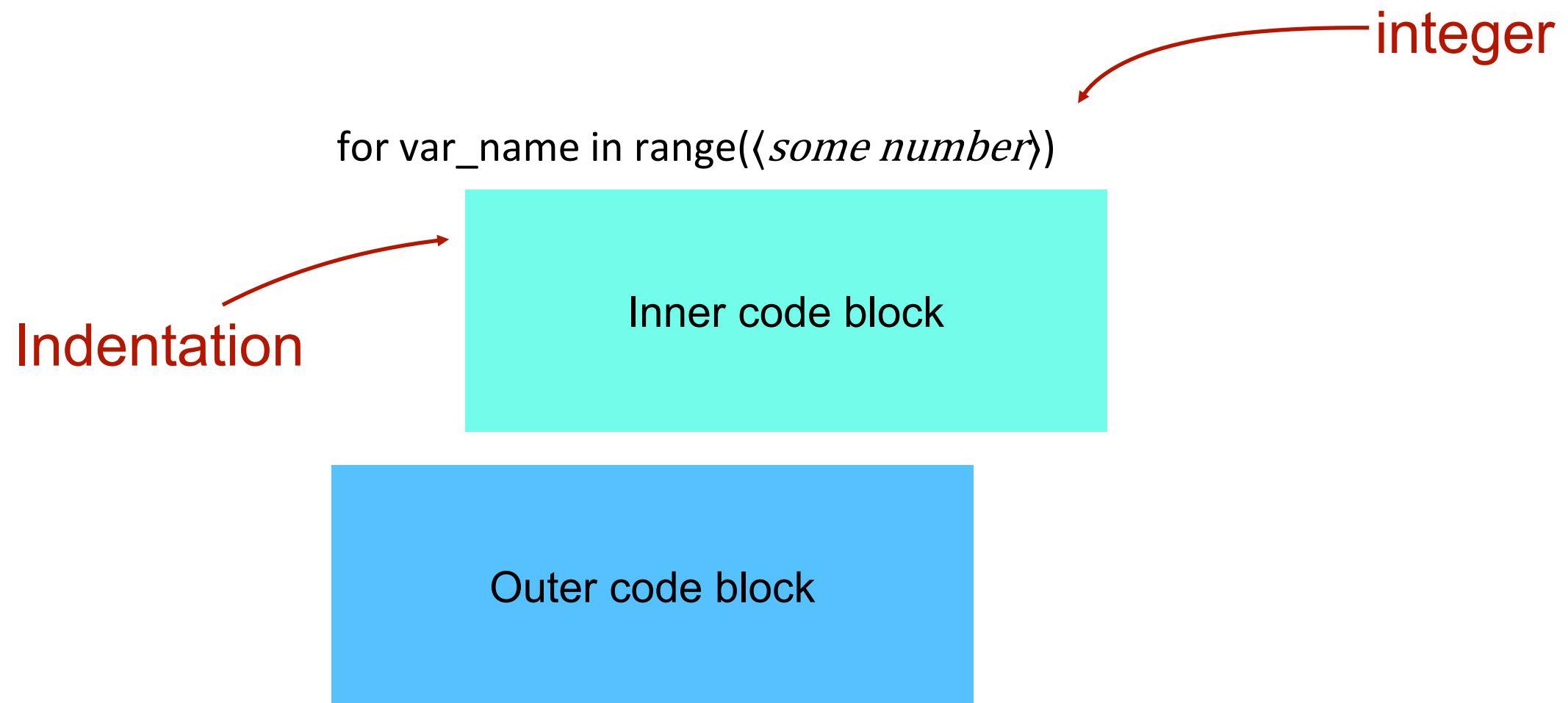
Statements encountered in the course so far :

While loop



For loops

Python allows repetition of code segments
using **while** / **for** loops



In the first iteration, `var_name` takes the value 0 and executes inner code block
it then takes the value 1 and executes inner code block, and continues,
until its value is equal to the number in `range(...)`

Control Flow using For loop

```
n = int(input("Enter number: "))
```

```
for i in range(n):  
    print(i)
```

range(n) is the set
 $\{0, 1, 2, \dots, n - 1\}$.
Empty set if $n \leq 0$

Control Flow using For loop

```
m = int(input("Enter starting number: "))  
n = int(input("Enter ending number: "))
```

```
for i in range(m, n):  
    print(i)
```

range(m, n) is the set
 $\{m, m + 1, m + 2, \dots, n - 1\}$.
Empty set if $n \leq m$

Control Flow using For loop

```
m = int(input("Enter starting number: "))  
n = int(input("Enter ending number: "))  
t = int(input("Enter increment: "))
```

```
for i in range(m, n, t):  
    print(i)
```

$\text{range}(m, n, t)$ is the set
 $\{m, m + t, m + 2t, \dots, n - 1\}$.
Empty set if $n \leq m$

Break statement in for loop

Exit loop if some condition holds
using **break** statement

```
mysum = 0

for i in range(5, 11, 2):
    mysum = mysum + i

    if (i == 9):
        break
    mysum = mysum + 1

print(mysum)
```

What is the output of this program?

21

Exercise: Primality testing using for loops

A positive number n is prime if the only numbers that divide it are 1 and n . Write a program **using for loops** that takes as input n , and outputs “Yes” if n is prime.

```
n = int(input("enter number: "))
i = 2
prime = True
while(i<n):
    if (n%i == 0):
        prime=False
        break
    i=i+1
if(prime):
    print(n,"is prime")
else:
    print(n,"is not prime")
```

```
n = int(input("enter number: "))
prime = True
for i in range(2,n):
    if (n%i == 0):
        prime=False
        break
if(prime):
    print(n,"is prime")
else:
    print(n,"is not prime")
```

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

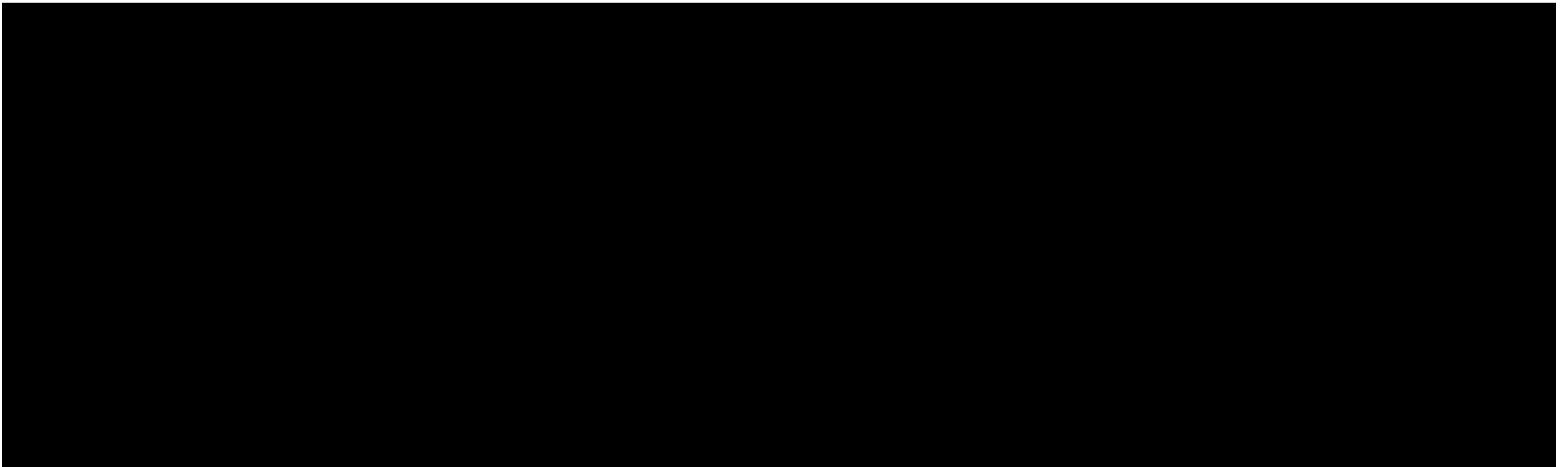
How many lines does this program output? What is the output?

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 0$

Output

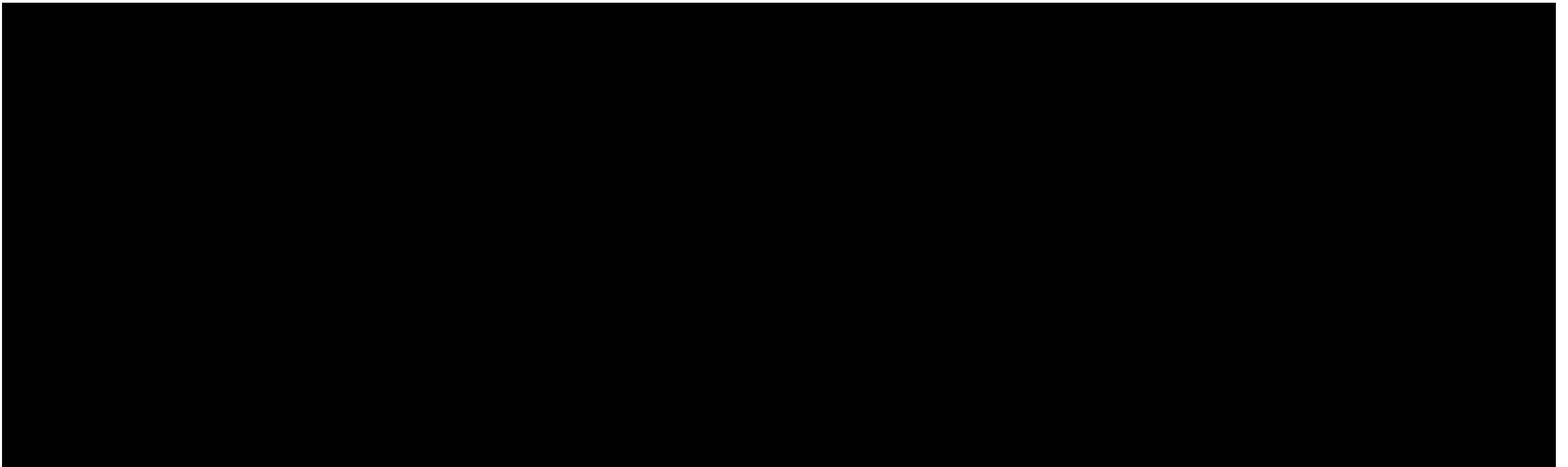


Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 0$

Output



Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
print(i)
```

$i = 0$

Output

0

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 1$

Output

0

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 1$

Output

0

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
print(i)
```

$i = 1$

Output

```
0  
1
```

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 2$

Output

```
0  
1
```

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 2$

Output

```
0  
1
```

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
print(i)
```

$i = 2$

Output

```
0  
1  
2
```

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 3$

Output

```
0  
1  
2
```


Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

$i = 3$

Output

```
0  
1  
2
```


Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

~~$i = 3$~~

Output

```
0  
1  
2
```

Guess the output

```
for i in range(10):  
    if i * i > 5:  
        break  
    print(i)
```

exits for loop

Output

```
0  
1  
2
```

Guess the output

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

How many lines does this program output? What is the output?

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

Output

Start of control-flow example

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0$

Output

Start of control-flow example

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 0$

Output

Start of control-flow example

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

~~$i = 0, j = 2$~~

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

$i = 1$

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

```
i = 2 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

~~$i = 2, j = 1$~~

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

```
i = 2 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

```
i = 2 j = 0
```

```
i = 2 j = 1
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

~~$i = 2, j = 2$~~

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

```
i = 2 j = 0
```

```
i = 2 j = 1
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

```
i = 2 j = 0
```

```
i = 2 j = 1
```

```
i = 2 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 0 j = 1
```

```
i = 0 j = 2
```

```
i = 1 j = 0
```

```
i = 1 j = 1
```

```
i = 1 j = 2
```

```
i = 2 j = 0
```

```
i = 2 j = 1
```

```
i = 2 j = 2
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

Output

```
Start of control-flow example
i = 0 j = 0
i = 0 j = 1
i = 0 j = 2
i = 1 j = 0
i = 1 j = 1
i = 1 j = 2
i = 2 j = 0
i = 2 j = 1
i = 2 j = 2
End of control-flow example
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

How many lines does this program output? What is the output?

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

Output

```
Start of control-flow example
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

Output

```
Start of control-flow example
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0$

Output

Start of control-flow example

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 0$

Output

Start of control-flow example

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
            # this is a comment
```

```
print("End of control-flow example")
```

$$i = 0, j = 0$$

Output

Start of control-flow example

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$$i = 0, j = 1$$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
            # this is a comment
```

```
print("End of control-flow example")
```

$$i = 0, j = 1$$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 0, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
# this is a comment
```

$i = 0$

```
print("End of control-flow example")
```

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

$i = 1$

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$$i = 1, j = 0$$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
        # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
            # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 1, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
# this is a comment
```

$i = 1$

```
print("End of control-flow example")
```

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
            # this is a comment
```

```
print("End of control-flow example")
```

$$i = 2, j = 0$$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):  
    for j in range(3):  
        if j == 1:  
            break
```

```
print("i =", i, "j =", j)
```

```
# this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 0$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

```
i = 2 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
    # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

```
i = 2 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
            # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

```
i = 2 j = 0
```

Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
            print("i =", i, "j =", j)
```

```
            # this is a comment
```

```
print("End of control-flow example")
```

$i = 2, j = 1$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

```
i = 2 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if j == 1:
```

```
            break
```

```
        print("i =", i, "j =", j)
```

```
# this is a comment
```

```
print("End of control-flow example")
```

$i = 2$

Output

```
Start of control-flow example
```

```
i = 0 j = 0
```

```
i = 1 j = 0
```

```
i = 2 j = 0
```


Guess the output

Program: easy_find_output.py

```
print("Start of control-flow example")

for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print("i =", i, "j =", j)
    # this is a comment

print("End of control-flow example")
```

Output

```
Start of control-flow example
i = 0 j = 0
i = 1 j = 0
i = 2 j = 0
End of control-flow example
```

Guess the output

Program: easy_find_output.py

```
for x in range(5):  
    for y in range(5):  
        if x + y == 6:  
            break  
        print("x=",x, "y=",y)
```

Output

```
x= 0 y= 0  
x= 0 y= 1  
x= 0 y= 2  
x= 0 y= 3  
x= 0 y= 4  
x= 1 y= 0  
x= 1 y= 1  
x= 1 y= 2  
x= 1 y= 3  
x= 1 y= 4  
x= 2 y= 0  
x= 2 y= 1  
x= 2 y= 2  
x= 2 y= 3  
x= 3 y= 0  
x= 3 y= 1  
x= 3 y= 2  
x= 4 y= 0  
x= 4 y= 1
```

Exercise : Program to list all primes numbers less than the input number

```
inp = int(input("Enter the upper bound: "))
```

```
for n in range(2, inp):
```

```
    flag = True
```

```
    for i in range(2, n):
```

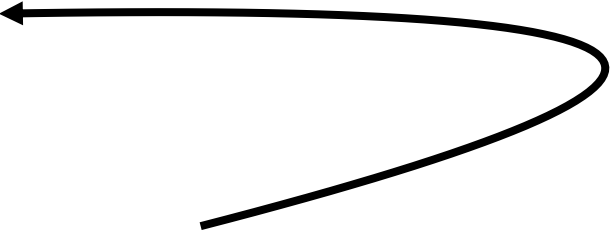
```
        if (n%i == 0):
```

```
            flag = False
```

```
            break
```

```
    if (flag):
```

```
        print(n, "is prime")
```



What happens if
flag = True
is moved outside the
outer for loop?

Exercise : Program to list all primes numbers less than the input number

```
inp = int(input("Enter the upper bound: "))
```

```
for n in range(2, inp):
```

```
    flag = True
```

```
    for i in range(2, n):
```

```
        if (n%i == 0):
```

```
            flag = False
```

```
            break
```

```
    if (flag):
```

```
        print(n, "is prime")
```

What happens if
flag = True
is moved outside the
outer for loop?

**At most two primes
printed. Why?**

Lecture 07

Strings and string manipulation

Strings and string manipulation

```
s = "Hello World" # can also use single quotes
```

```
print(s)
```

```
s1 = s + '!' 
```

```
print (s1)
```

```
s2 = s + "!"*2
```

```
print (s2)
```

Output

```
Hello World
Hello World!
Hello World!!
```

Operations on strings seen so far

`s1 + s2` : concatenates strings

`s * 2` : two copies of s

`s1 == s2` : `True` if both strings are equal (case sensitive)

`print(s)` : prints the string s

`type(s)` : prints the type of s

`int(s)` : converts to integer

(error if s cannot be converted to integer)

`float(s)` : converts to float

(error if s cannot be converted to float)

Accessing characters of string: Indexing

(All indexing in Python starts at 0)

The i^{th} character of string s can be accessed using
 $s[i]$

```
s = "Hello World"
print ("The third character is", s[2])
```

Output

```
The third character is l
```


Accessing characters of string: Indexing

(All indexing in Python starts at 0)

The i^{th} character of string s can be accessed using
 $s[i]$

What is the output of the following program?

```
s = "Hello World"  
  
print (s[11])
```

Output

```
IndexError: string index out of range
```

Accessing characters of string: Reverse Indexing

The i^{th} character of string s from the end can be accessed using $s[-i]$

```
s = "Hello World"

print ("The last character is", s[-1])
```

Output

```
The last character is d
```

Length of string

The length of string `s` can be accessed using **`len(s)`**

```
s = "Hello World"  
  
print(len(s))
```

Output

```
11
```

Slicing a string

(All indexing in Python starts at 0)

Substring starting at position `start` and ending at `end-1`
`s[ start: end ]`

```
s = "Hello World"
print("s[2:5] is", s[2:5])
```

Output

```
s[2:5] is llo
```

Slicing a string

(All indexing in Python starts at 0)

Substring starting from position `start` to end of string
`s[ start : ]`

```
s = "Hello World"
print ("s[2:] is", s[2:])
```

Output

```
s[2:] is llo World
```

Example: extract year from dd/mm/yyyy

Write a program that takes as input a date in the **dd/mm/yyyy** format, and outputs the year.

```
s = input("enter date in dd/mm/yyyy format: ")  
    s[-4:] # t = s[6:] or t = s[6:10] also works  
t = _____  
print(t)
```

Example: count number of vowels

Write a program that takes as input a five-letter word and outputs the number of vowels in the word.

```
s = input("enter a five-letter word: ")

count = 0

if ((s[0] == 'a') or (s[0] == 'e') or (s[0] == 'i') or (s[0] == 'o') or (s[0] == 'u')):
    count = count + 1

if ((s[1] == 'a') or (s[1] == 'e') or (s[1] == 'i') or (s[1] == 'o') or (s[1] == 'u')):
    count = count + 1

if ((s[2] == 'a') or (s[2] == 'e') or (s[2] == 'i') or (s[2] == 'o') or (s[2] == 'u')):
    count = count + 1

if ((s[3] == 'a') or (s[3] == 'e') or (s[3] == 'i') or (s[3] == 'o') or (s[3] == 'u')):
    count = count + 1

if ((s[4] == 'a') or (s[4] == 'e') or (s[4] == 'i') or (s[4] == 'o') or (s[4] == 'u')):
    count = count + 1

print("The number of vowels is", count)
```

String equality is case-sensitive.

Would give incorrect answer if some characters in word are uppercase

Example: count number of vowels

s.lower() : converts all alphabets to lowercase

s.upper() : converts all alphabets to uppercase

```
t = input("enter a five-letter word: ")
s = t.lower()

count = 0

if ((s[0] == 'a') or (s[0] == 'e') or (s[0] == 'i') or (s[0] == 'o') or (s[0] == 'u')):
    count = count + 1

if ((s[1] == 'a') or (s[1] == 'e') or (s[1] == 'i') or (s[1] == 'o') or (s[1] == 'u')):
    count = count + 1

if ((s[2] == 'a') or (s[2] == 'e') or (s[2] == 'i') or (s[2] == 'o') or (s[2] == 'u')):
    count = count + 1

if ((s[3] == 'a') or (s[3] == 'e') or (s[3] == 'i') or (s[3] == 'o') or (s[3] == 'u')):
    count = count + 1

if ((s[4] == 'a') or (s[4] == 'e') or (s[4] == 'i') or (s[4] == 'o') or (s[4] == 'u')):
    count = count + 1

print("The number of vowels is", count)
```

We are repeating the same code five times. A better way of doing this?

Exercise: output of the code

```
string = "abcde"

for i in range(len(string) + 1):
    print(string[i])
```

Output

a
b
c
d
e

ERROR!

Traceback (most recent call last):

File "<main.py>", line 4, in <module>
IndexError: string index out of range

For loops can also iterate over characters of string

```
ss = "abcde"  
  
for i in ss:  
    print(i)
```

Output

a
b
c
d
e

Program to count the number of spaces in a string

```
s = input("enter string: ")  
  
count = 0  
  
for i in s :  
    if(i == ' '):  
        count = count + 1  
  
print("number of spaces is", count)
```

Program to detect repeated characters in a string

```
flag = False

s = input("enter string: ")

for i in range(len(s)):
    for j in range(i + 1, len(s)):
        if (s[i] == s[j]):
            flag = True

if flag:
    print("The string has repeated characters.")
else:
    print("All characters are unique.")
```

End of For loops

See [Guttag] Section 2.6

End of [Guttag] Chapter 2

Review lecture

Program to print the first n even numbers

Using while loop:

Using for loop:

Program to add the first n even numbers

Using while loop:

Using for loop:

Primality testing

A positive number n is prime if the only numbers that divide it are 1 and n . Write a program **using for loops** that takes as input n , and outputs “Yes” if n is prime.

```
n = int(input("enter number: "))

prime = True

for i in range(2,n):

    if (n%i == 0):
        prime=False
        break

if(prime):
    print(n,"is prime")
else:
    print(n,"is not prime")
```

Program that prints the first prime number greater than 10000

```
n = 10000
found_prime = False

while(found_prime == False) :

    flag = True

    for i in range(2, n) :
        if (n%i == 0):
            flag = False
            break

    if (flag == True):
        print(n, "is prime")
        found_prime = True
        break

    n = n + 1
```

Program that prints the prime factorisation of a number



Program to detect if first character appears multiple times in a string

Program to detect repeated characters in a string

```
flag = False

s = input("enter string: ")

for i in range(len(s)):
    for j in range(i + 1, len(s)):
        if (s[i] == s[j]):
            flag = True

if flag:
    print("The string has repeated characters.")
else:
    print("All characters are unique.")
```